SPE UMaT Chapter Website

Frontend Developer Guide

React + TypeScript: public site, member pages and admin portal

Project	SPE UMaT Chapter website
Document	Frontend Developer Guide
Version	1.0, final
Audience	Frontend developer
Companion	Backend Developer Guide v1.0. Sections 4 and 13 and the appendices are identical in both guides

1. Purpose and scope

This document is the complete specification for the frontend of the SPE UMaT Chapter website. It is written for the frontend developer and is to be followed as written. The backend developer works from the companion Backend Developer Guide. Both guides contain the same API contract (section 4), database schema (Appendix A) and rulings (section 13), so the two codebases will match without further discussion.

The frontend is one React + TypeScript application deployed once on Vercel. It has three areas:

Area	Contents	Login
Public website	Homepage, About, Events, News, Media, Leaderboard	None
Member pages	Registration with email verification, merit lookup, event check-in	None. Members have no accounts
Admin section	Executive portal: overview, registrations, events and QR, news, attendance, merit, media and content, messages, audit log, admins	Admins only, through Supabase Auth

Outside the frontend's responsibility

- The database, RLS policies, Storage rules and migrations.
- Edge Functions: registration, verification, merit lookup, check-in, QR token signing, email, CAPTCHA verification, ID generation, distance and location checks.
- Every secret. The frontend holds only the public Supabase URL, the anon key, the site URL and the CAPTCHA site key.

1.1 Requirement traceability

#	Committee requirement	Where it is covered in this guide
1	Homepage	6.1
2	About page	6.2
3	Website research and inspiration	15 (design research note)
4	Member registration and 6-digit membership ID	7.1 to 7.3

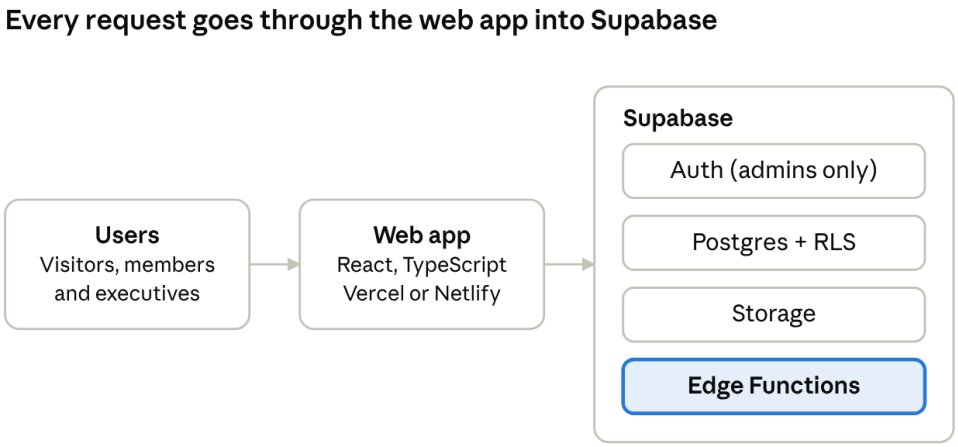
#	Committee requirement	Where it is covered in this guide
5	Merit lookup by membership ID	7.4
6	Administrative section and registrations export	8.1 to 8.4
7	News, media and content management	6.3 to 6.6, 8.8 to 8.10
8	Merit Tracker and event QR check-in	7.5, 8.5 to 8.7

2. Governing rules

Both developers work against the same contract. These five rules do not change without both developers agreeing.

1. **No accounts for visitors or members.** Their three actions (register, look up merit, check in) go through Edge Function endpoints only. The frontend never reads the `members` table directly.
2. **Only admins sign in**, through Supabase Auth. Admin screens read and write data under Row Level Security (RLS).
3. **The backend owns every secret** (service role key, CAPTCHA secret, Resend key, device-hash pepper). The frontend holds only the public URL and anon key.
4. **The schema in the appendix is the single source of truth.** Any change is agreed by both developers before merging.
5. **Validation and role checks on the client are UX only.** Server-side validation in the Edge Functions and RLS in the database are the real enforcement.

Architecture



Every request goes through the web app into Supabase. Public content is read with `supabase-js` and the anon key. Member actions call Edge Functions. Admin screens use `supabase-js` under RLS and call admin Edge Functions with the admin's session token.

3. Technology and project setup

3.1 Stack

Layer	Choice
Framework	React 18 with TypeScript, built with Vite
Routing	React Router 6
Styling	Tailwind CSS, themed with design tokens (3.4)
State	Zustand for check-in, registration and auth state
Backend client	`@supabase/supabase-js` for reads, admin writes, Auth and Storage; `fetch` for Edge Functions
Forms and validation	React Hook Form with Zod
CAPTCHA	Cloudflare Turnstile widget (`@marsidev/react-turnstile`)
Markdown	`react-markdown` with `rehype-sanitize`
Charts	Recharts, admin Overview only
QR codes	`qrcode.react` on the admin QR screen. Members scan with the phone's own camera, so no scanner library is used
Export	`papaparse` for CSV and `xlsx` (SheetJS) for Excel, in the browser
Token decoding	`jwt-decode` to read the `app_roles` claim
Hosting	Vercel

3.2 Environment variables

```
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<anon key>
VITE_SITE_URL=https://<production domain>     # used inside QR codes and Auth redirects
VITE_CAPTCHA_SITE_KEY=<Turnstile site key>
```

Create one set for staging and one for production in Vercel. Only public values belong here. A service role key or any other secret must never appear in the frontend code, the bundle or the repository.

3.3 Project structure

```
src/
  app/            router.tsx, providers.tsx, layouts/PublicLayout.tsx, layouts/AdminLayout.tsx
  config/         programmes.ts, site.ts
  pages/
    public/       Home, About, Events, EventDetail, News, NewsDetail, Media, Leaderboard
    member/       Register, Verify, RegisterSuccess, Merit, CheckInScan, CheckIn
    admin/        Login, AcceptInvite, ResetPassword, Overview, Registrations, Events,
                  EventForm, EventQR, News, NewsForm, Attendance, MeritTracker,
                  MediaContent, Messages, AuditLog, Admins
  components/     ui, forms, tables, layout, markdown, gallery, calendar, qr
  lib/            supabase.ts, api.ts, errorMessages.ts, device.ts, geo.ts, roles.ts,
                  image.ts (resize), export.ts, format.ts (Africa/Accra dates)
  schemas/        one Zod schema file per form
  store/          authStore.ts, registrationStore.ts, checkinStore.ts
  types/          api-types.ts (Appendix B), db-types.ts (generated from the schema)
```

3.4 Design tokens and branding

Define colours, fonts and radii as Tailwind tokens in `tailwind.config.ts`: `brand-primary`, `brand-secondary`, `brand-accent`, `surface`, `text`. Use these defaults until the chapter's brand assets are supplied (prerequisite P8), then replace the values in one place:

```
brand-primary:   #0B3C6F     brand-secondary: #1F7A8C     brand-accent: #F2A900
surface:         #FFFFFF     text:            #1A2433
Fonts: system font stack (no web fonts, to save mobile data).
```

3.5 Deployment configuration

```
// vercel.json
{
  "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }],
  "headers": [{
    "source": "/(.*)",
    "headers": [
      { "key": "X-Content-Type-Options", "value": "nosniff" },
      { "key": "Referrer-Policy", "value": "strict-origin-when-cross-origin" },
      { "key": "Permissions-Policy", "value": "geolocation=(self), camera=(), microphone=()" }
    ]
  }]
}
```

`Permissions-Policy: geolocation=(self)` is required so the check-in page can request the phone's position. The site is served over HTTPS only, which the Geolocation API also requires.

3.6 Supabase client and API client

```
// lib/supabase.ts
import { createClient } from '@supabase/supabase-js';
export const supabase = createClient(
  import.meta.env.VITE_SUPABASE_URL,
  import.meta.env.VITE_SUPABASE_ANON_KEY,
  { auth: { persistSession: true, autoRefreshToken: true, detectSessionInUrl: true } }
);

// lib/api.ts
import type { ApiResponse, ErrorCode } from '../types/api-types';
import { MESSAGES } from './errorMessages';
import { supabase } from './supabase';

export class ApiError extends Error {
  constructor(public code: ErrorCode, message: string,
              public details?: { field: string; message: string }[],
              public retryAfter?: number) { super(message); }
}

export async function callFn<Res>(name: string, body: object = {}, accessToken?: string):
Promise<Res> {
  const headers: Record<string, string> = {
    'Content-Type': 'application/json',
    apikey: import.meta.env.VITE_SUPABASE_ANON_KEY,
  };
  if (accessToken) headers.Authorization = 'Bearer ' + accessToken;
  let json: ApiResponse<Res>;
  try {
    const res = await fetch(import.meta.env.VITE_SUPABASE_URL + '/functions/v1/' + name, {
      method: 'POST', headers, body: JSON.stringify(body), signal: AbortSignal.timeout(15000),
    });
    json = await res.json();
  } catch {
    throw new ApiError('SERVER_ERROR', MESSAGES.SERVER_ERROR);
  }
  if (!json.ok) {
    const e = json.error;
```

```
    throw new ApiError(e.code, MESSAGES[e.code] ?? e.message, e.details, e.retry_after_seconds);
  }
  return json.data;
}

// Admin functions: always attach the current session token.
export async function adminCall<Res>(name: string, body: object = {}): Promise<Res> {
  const { data } = await supabase.auth.getSession();
  if (!data.session) throw new ApiError('UNAUTHORIZED', MESSAGES.UNAUTHORIZED);
  return callFn<Res>(name, body, data.session.access_token);
}
```

`lib/errorMessages.ts` holds the "Message shown to the user" column of the error table in section 4.2 as a `Record<ErrorCode, string>`. Components display `error.message` from `ApiError`. On `UNAUTHORIZED` from any admin call, the app signs the admin out and redirects to `/admin/login`.

4. API contract

> **Authoritative** This section is identical in the Backend Guide. Build to it exactly. A change requires a written amendment to both guides.

4.1 Conventions

All functions follow the same conventions. They are fixed and apply to both codebases.

Item	Rule
URL	`https://<project-ref>.supabase.co/functions/v1/<function_name>`
Method and body	`POST` with a JSON body, `Content-Type: application/json`. Functions with no request fields take `{}`
Headers, public functions	`apikey: <anon key>`. No `Authorization` header. These functions are deployed with JWT verification off (`--no-verify-jwt`) because visitors have no session
Headers, admin functions	`apikey: <anon key>` and `Authorization: Bearer <admin access token>` from the current Supabase session. JWT verification stays on, and each function also checks the role in code
Success response	HTTP 200. `{ "ok": true, "data": { ... } }`. Functions with no response fields return `"data": {}`
Error response	HTTP status from the error table. `{ "ok": false, "error": { "code": "...", "message": "...", "details": [...], "retry_after_seconds": 30 } }`. `details` and `retry_after_seconds` appear only when relevant
What the client switches on	The `error.code` value, never the HTTP status or the message text
Field names	`snake_case` in every request and response
Timestamps	ISO 8601 strings in UTC. The site displays them in the `Africa/Accra` time zone
CORS	Allowed origins are the production site URL, plus `http://localhost:5173` and the Vercel preview URL on staging only. `OPTIONS` requests are answered by every function
Timeout	The client aborts a call after 15 seconds and shows the `SERVER_ERROR` message

4.2 Error codes

Every function returns one of these codes on failure. Handle each code a function lists. Show the message in the last column unless stated otherwise.

Code	HTTP	Raised when	Message shown to the user
`VALIDATION_ERROR`	400	Input fails validation. `error.details` lists `field` and `message` pairs	Show each message beside its field
`CAPTCHA_FAILED`	400	CAPTCHA token missing or rejected	Please complete the verification and try again.
`CODE_INVALID`	400	Wrong email verification code	That code is not correct.
`LOCATION_DENIED`	400	No position supplied, permission denied, or accuracy worse than 100 m	Location is required. Allow location for this site, then scan the QR code again.
`UNAUTHORIZED`	401	Admin call without a valid session	Redirect to the admin login page
`FORBIDDEN`	403	Signed-in admin lacks the required role	You do not have permission to do this.
`CHECKIN_CLOSED`	403	Event unpublished, outside the check-in window, or check-in closed by an admin	Check-in is closed for this event.
`OUTSIDE_AREA`	403	Phone is farther from the venue than the event radius	You do not appear to be at the event. Move closer and try again.
`ID_NOT_FOUND`	404	No member has this ID. One generic response for every unknown ID	ID not found.
`NOT_FOUND`	404	Admin call refers to a record that does not exist	This item no longer exists.
`DUPLICATE_REGISTRATION`	409	Email or phone already registered	This email or phone number is already registered. Please contact the chapter executives.
`ALREADY_CHECKED_IN`	409	Member already checked in to this event. No extra points	You are already checked in to this event.
`DEVICE_ALREADY_USED`	409	This phone already checked someone in for this event	This phone has already been used to check in for this event.
`EVENT_HAS_CHECKINS`	409	Admin tried to delete an event that has check-ins	This event has check-ins and cannot be deleted. Unpublish it instead.
`LAST_TECHNICAL_HEAD`	409	Attempt to remove the last Technical Head, or the caller's own Technical Head role	At least one Technical Head must remain.
`CODE_EXPIRED`	410	Verification code older than 10 minutes, or no pending registration	That code has expired. Request a new one or register again.
`QR_EXPIRED`	410	QR token outside its life, invalid signature, or event secret reset	This QR code has expired. Scan the screen again.
`SESSION_INVALID`	410	Check-in session missing, used, expired (3 minutes), from another device, or over 5 attempts	Your check-in session has ended. Scan the QR code again.
`RATE_LIMITED`	429	Too many requests. `error.retry_after_seconds` is set	Too many attempts. Please wait and try again.

Code	HTTP	Raised when	Message shown to the user
`TOO_MANY_ATTEMPTS`	429	More than 5 wrong verification codes for one registration	Too many wrong codes. Please register again.
`SERVER_ERROR`	500	Unexpected failure	Something went wrong. Please try again.

4.3 Public functions (no login)

register_member

Access: Public. JWT verification off. Validates the registration, rejects duplicates, stores a pending registration and emails a 6-digit verification code.

Request field	Type	Rules
`first_name`	string	Required. 1 to 50 characters after trimming
`last_name`	string	Required. 1 to 50 characters after trimming
`programme`	string	Required. 2 to 100 characters. A value from the programme list, or free text when “Other” is chosen
`year_of_study`	string	Required. One of `Year 1`, `Year 2`, `Year 3`, `Year 4`, `Postgraduate`
`email`	string	Required. Valid email, maximum 254 characters. Stored in lower case
`phone`	string	Required. Ghana format (`0XXXXXXXXX`, `233XXXXXXXXX`, `+233XXXXXXXXX`) or international E.164 (`+` and 8 to 15 digits). Stored as E.164
`show_on_leaderboard`	boolean	Optional. Default `false`
`captcha_token`	string	Required. Token from the Turnstile widget
`website`	string	Honeypot. Always send an empty string. A non-empty value is discarded silently

Response field	Type	Description
`email`	string	The normalised email the code was sent to
`expires_in_seconds`	integer	600
`resend_available_in_seconds`	integer	60

Errors: `VALIDATION_ERROR`, `CAPTCHA_FAILED`, `DUPLICATE_REGISTRATION`, `RATE_LIMITED`.

verify_email

Access: Public. JWT verification off. Checks the emailed code, creates the member, generates the membership ID, returns it once and emails it.

Request field	Type	Rules
`email`	string	Required. The email used to register
`code`	string	Required. Exactly 6 digits

Response field	Type	Description
`member_code`	string	The new 6-digit membership ID. Returned only by this call

Response field	Type	Description
`first_name`	string	—

Errors: `VALIDATION_ERROR`, `CODE_INVALID`, `CODE_EXPIRED`, `TOO_MANY_ATTEMPTS`, `DUPLICATE_REGISTRATION`, `RATE_LIMITED`.

resend_code

Access: Public. JWT verification off. Sends a new verification code for a pending registration. Allowed after a 60-second cooldown and at most 3 times per registration.

Request field	Type	Rules
`email`	string	Required

Response field	Type	Description
`expires_in_seconds`	integer	600
`resend_available_in_seconds`	integer	60

Errors: `VALIDATION_ERROR`, `CODE_EXPIRED` (no pending registration), `RATE_LIMITED`.

merit_lookup

Access: Public. JWT verification off. Returns a member's first name, total points and merit history. Returns no contact details.

Request field	Type	Rules
`member_code`	string	Required. Exactly 6 digits, first digit 1 to 9

Response field	Type	Description
`first_name`	string	—
`total_points`	integer	Sum of all ledger rows
`attendance`	array	Each item: `event_title` string, `event_date` ISO timestamp, `points` integer, `checked_in_at` ISO timestamp. Newest first
`adjustments`	array	Each item: `points` integer, `reason` string, `created_at` ISO timestamp. Newest first

Errors: `VALIDATION_ERROR`, `ID_NOT_FOUND`, `RATE_LIMITED`.

start_check_in

Access: Public. JWT verification off. Validates the scanned QR token, the phone's location and the event window, then opens a one-time check-in session bound to the phone.

Request field	Type	Rules
`event_token`	string	Required. The token from the QR address
`device_token`	string	Required. UUID generated by the browser and kept in local storage (see Frontend section 7.5)
`location`	object	Required. `lat` number (-90 to 90), `lng` number (-180 to 180), `accuracy` number in metres (greater than 0)

Response field	Type	Description
session_id	uuid	Valid once, for 3 minutes, from this device only
expires_at	string	ISO timestamp, 3 minutes after the scan
event_title	string	—
points	integer	Points this check-in will award

Errors: VALIDATION_ERROR, QR_EXPIRED, CHECKIN_CLOSED, LOCATION_DENIED, OUTSIDE_AREA, RATE_LIMITED.

check_in

Access: Public. JWT verification off. Completes a check-in: validates the session, device, location, event window and member ID, then records attendance and awards points in one transaction.

Request field	Type	Rules
session_id	uuid	Required
member_code	string	Required. Exactly 6 digits, first digit 1 to 9
device_token	string	Required. UUID generated by the browser and kept in local storage (see Frontend section 7.5)
location	object	Required. lat number (-90 to 90), lng number (-180 to 180), accuracy number in metres (greater than 0)

Response field	Type	Description
first_name	string	—
points_awarded	integer	—
total_points	integer	New total
event_title	string	—

Errors: VALIDATION_ERROR, SESSION_INVALID, CHECKIN_CLOSED, LOCATION_DENIED, OUTSIDE_AREA, ID_NOT_FOUND, ALREADY_CHECKED_IN, DEVICE_ALREADY_USED, RATE_LIMITED.

submit_contact

Access: Public. JWT verification off. Stores a Contact Us message and notifies the executives.

Request field	Type	Rules
name	string	Required. 2 to 100 characters
email	string	Required. Valid email
message	string	Required. 10 to 2000 characters
captcha_token	string	Required
website	string	Honeypot. Always send an empty string

Response: data: {}.

Errors: VALIDATION_ERROR, CAPTCHA_FAILED, RATE_LIMITED.

4.4 Admin functions (admin session required)

qr_token

Access: Organization Head, Technical Head. Returns the current signed QR token for an event. Writes nothing to the database.

Request field	Type	Rules
`event_id`	uuid	Required

Response field	Type	Description
`token`	string	Place in `VITE_SITE_URL + "/checkin/" + token`
`refresh_in_seconds`	integer	Seconds until the next 30-second window starts. Call again then

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

set_checkin_state

Access: Organization Head, Technical Head. Opens or closes check-in for an event.

Request field	Type	Rules
`event_id`	uuid	Required
`is_active`	boolean	Required

Response field	Type	Description
`is_active`	boolean	—

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

reset_event_secret

Access: Organization Head, Technical Head. Replaces the event's secret. Every outstanding QR code and check-in session stops working immediately.

Request field	Type	Rules
`event_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

manual_check_in

Access: Organization Head, Technical Head. Checks a member in by hand. Awards the event's points. No location or device check.

Request field	Type	Rules
`event_id`	uuid	Required
`member_code`	string	Required. Exactly 6 digits, first digit 1 to 9

Response field	Type	Description
`first_name`	string	—
`points_awarded`	integer	—
`total_points`	integer	—

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `ID_NOT_FOUND`, `ALREADY_CHECKED_IN`.

remove_check_in

Access: Organization Head, Technical Head. Removes a check-in and writes a reversing points row.

Request field	Type	Rules
`attendance_id`	uuid	Required
`reason`	string	Required. 3 to 200 characters

Response field	Type	Description
`total_points`	integer	The member's new total

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

adjust_merit

Access: Organization Head, Technical Head. Adds a manual points adjustment to the ledger.

Request field	Type	Rules
`member_code`	string	Required. Exactly 6 digits, first digit 1 to 9
`points`	integer	Required. Non-zero, from -100 to 100
`reason`	string	Required. 3 to 200 characters. Visible to the member in merit lookup

Response field	Type	Description
`total_points`	integer	—

Errors: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `ID_NOT_FOUND`.

log_export

Access: Organization Head, Technical Head. Records a registrations export in the audit log. The frontend calls it before building the file and builds the file only if it succeeds.

Request field	Type	Rules
`format`	string	Required. `csv` or `xlsx`
`row_count`	integer	Required. Rows about to be exported

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`.

delete_event

Access: Organization Head, Technical Head. Deletes an event that has no check-ins, with its cover image and linked rows.

Request field	Type	Rules
`event_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `EVENT_HAS_CHECKINS`.

delete_news_post

Access: Communications Head, Technical Head. Deletes a news post and its cover image.

Request field	Type	Rules
`post_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

delete_media_item

Access: Communications Head, Technical Head. Deletes a media item and, for pictures, its file in Storage.

Request field	Type	Rules
`media_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

delete_member

Access: Technical Head. Permanently deletes a member with their attendance and points. Used to honour a data-removal request.

Request field	Type	Rules
`member_id`	uuid	Required

Response: `data: {}`.

Errors: `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`.

list_admins

Access: Technical Head. Lists admin accounts with their roles.

Request: empty object.

Response field	Type	Description
`admins`	array	Each item: `user_id` uuid, `email` string, `roles` array of role names, `last_sign_in_at` ISO timestamp or null

Errors: `UNAUTHORIZED`, `FORBIDDEN`.

invite_admin

Access: Technical Head. Creates an admin account, assigns a role and emails an invitation link to set a password.

Request field	Type	Rules
`email`	string	Required
`role`	string	Required. `communications_head`, `organization_head` or `technical_head`

Response field	Type	Description
`user_id`	uuid	—

Errors: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`.

set_user_role

Access: Technical Head. Adds or removes one role for an admin. Removing all roles disables the admin's access.

Request field	Type	Rules
`user_id`	uuid	Required
`role`	string	Required. One of the three role names
`action`	string	Required. `add` or `remove`

Response field	Type	Description
`roles`	array	The admin's roles after the change

Errors: `VALIDATION_ERROR`, `UNAUTHORIZED`, `FORBIDDEN`, `NOT_FOUND`, `LAST_TECHNICAL_HEAD`.

The TypeScript types for every request and response are in Appendix B. Copy them into `src/types/api-types.ts`.

5. Direct data access (supabase-js)

Everything that is not an Edge Function call goes through `supabase-js` under RLS. The tables and views below are the only ones the frontend touches.

5.1 Public reads (anon key)

Source	Query	Used by
`site_content`	select `key, body` where key in the keys needed	Home, About, footer
`events_public`	select; filter and order by `starts_at`; single event by `slug`	Home, Events, Event detail, Media calendar
`news_posts`	select where `is_published = true`, order by `published_at` desc; single post by `slug`	Home, News
`media_items`	select where `is_published = true`; filter by `type`, `category`, `event_id`	Home, Media, Event detail
`leaderboard`	select order by `rank` limit 20	Leaderboard

Never query `events`, `members`, `attendance` or `event_checkin_tokens` from a public page.

5.2 Admin reads (admin session)

Source	Role	Used by
`admin_registrations`	Organization Head, Technical Head	Registrations, Overview, Merit Tracker
`admin_attendance`	Organization Head, Technical Head	Attendance, Overview
`admin_merit`	Organization Head, Technical Head	Merit Tracker
`events`	All three roles	Events screens
`event_checkin_tokens` (`is_active` only)	Organization Head, Technical Head	Event screen check-in toggle
`contact_messages`	Communications Head, Technical Head	Messages, Overview
`audit_logs`	Technical Head	Audit log
`news_posts`, `media_items`, `site_content`	Communications Head, Technical Head	News, Media and Content

5.3 Admin writes (admin session, RLS enforced)

Table	Operations	Role
`site_content`	update `body`	Communications Head, Technical Head
`events`	insert, update. Communications Head cannot change venue, radius, points or check-in window; the database refuses	All three roles
`news_posts`	insert, update (including `is_published`)	Communications Head, Technical Head
`media_items`	insert, update	Communications Head, Technical Head
`contact_messages`	update `is_read`	Communications Head, Technical Head

Deleting rows, adjusting points, checking members in, resetting secrets, opening or closing check-in, exports and role changes are done only through the admin functions in section 4.4.

5.4 Storage

Item	Rule
Bucket	`site-media`, public read
Paths	`events/<uuid>.jpg` for event covers, `news/<uuid>.jpg` for news covers, `gallery/<uuid>.jpg` for pictures
Public URL	`supabase.storage.from("site-media").getPublicUrl(path)`
Upload	Directly from the browser under RLS. Event covers: any admin role. News covers and gallery: Communications Head, Technical Head
Replacing a cover image	Upload the new file, save its URL on the row, then remove the old file with `storage.remove`

Item	Rule
Deleting	Gallery pictures, news posts and events are deleted through their delete functions, which also remove the files

6. Public pages

No public route requires a login. All dates and times are displayed in `Africa/Accra`.

Route	Page	Section
/	Homepage	6.1
/about	About	6.2
/events, /events/:slug	Events and event detail	6.3
/news, /news/:slug	News and post detail	6.4
/media	Media	6.5
/leaderboard	Leaderboard	6.6
/register, /register/verify, /register/success	Registration	7.1 to 7.3
/merit	Merit lookup	7.4
/checkin/:eventToken, /checkin	Event check-in	7.5

6.1 Homepage (/)

Sections, in order:

1. **Hero:** chapter name, one-line description, buttons “Become a member” (`/register`) and “Check my merit” (`/merit`).
2. **About SPE and the chapter:** `site_content.home_intro` rendered as Markdown, with a link to `/about`.
3. **Upcoming events:** up to 3 events from `events_public` where `ends_at` is in the future, ordered by `starts_at`. Each card has the cover image, title, date and a link to `/events/:slug`. When there are none, show “No upcoming events right now. Check back soon.”
4. **Latest news:** the 3 newest published posts, linking to `/news/:slug`.
5. **Photos:** the 8 newest published photos from `media_items` (type `photo`), lazy-loaded, linking to `/media`.
6. **Our executives:** published photos with category `executive`; the `title` field holds “Name, Position”.
7. **Contact Us:** `site_content.contact_info` beside the contact form (name, email, message). Submit calls `submit_contact`. Include the Turnstile widget and the hidden `website` field. Show the notice from section 9.3. On success replace the form with “Thank you. Your message has been sent.”

6.2 About (/about)

- Sections from `site_content`: `about_mission` (SPE mission), `about_vision` (vision), `about_benefits` (benefits of membership). Also a fixed section on the SPE UMaT Chapter, using `home_intro`.
- End with a call to action linking to `/register`.

6.3 Events (/events, /events/:slug)

- **List:** tabs “Upcoming” (ends in the future, soonest first) and “Past” (newest first). 12 per page with “Load more”. Card: cover image, title, date, location.

- **Detail:** cover image, title, date and time, location, description rendered as Markdown. Below it, a gallery of photos where `media_items.event_id` equals the event. An unknown or unpublished slug shows a 404 page.
- Read `events_public` only.

6.4 News (/news, /news/:slug)

- **List:** 9 per page, newest first, with "Load more". Card: cover image (a branded placeholder if none), title, date, first 140 characters of the body as plain text.
- **Detail:** title, date, cover image, body rendered as Markdown with `rehype-sanitize`. Unknown or unpublished slug shows a 404 page.

6.5 Media (/media)

Section	Content
Videos	`media_items` where `type = youtube`. Embed with the `youtube-nocookie.com/embed/<id>` URL, lazy-loaded; extract the id from `youtube.com/watch?v=`, `youtu.be/` or `youtube.com/embed/` links
Recordings	`type = recording`: title and an external link button
Gallery	`type = photo`. Grid with a category filter (All, Event, Activity, Member, Executive) and a full-screen lightbox. Lazy-load every image
Event calendar	Month view built from `events_public`. Each event links to its detail page

6.6 Leaderboard (/leaderboard)

- Table of rank, display name and points for the top 20 rows of `leaderboard`. Explain under the table: "Members appear here only if they chose to when registering."
- Add "Leaderboard" to the main navigation and footer.

7. Member flows

7.1 Registration (/register)

Field	Control	Rule
First name, last name	Text	Required, 1 to 50 characters
Programme	Select from `config/programmes.ts`. "Other" reveals a text field	Required, 2 to 100 characters
Year of study	Select: Year 1, Year 2, Year 3, Year 4, Postgraduate	Required
Email	Email	Required, valid
Phone	Tel	Required. Ghana or international format (section 9.1)
Leaderboard	Checkbox, unticked, label from section 9.3	Optional
Verification	Turnstile widget	Required
`website`	Hidden honeypot: positioned off-screen, `tabIndex=-1`, `autoComplete="off"`, `aria-hidden`	Always send an empty string unless a bot fills it

1. Show the data-protection notice under the form (section 9.3).
2. On submit call `register_member`. While waiting, disable the button and show a spinner.
3. On success, store `email` and the two countdown values in `registrationStore` and navigate to `/register/verify`.
4. On `DUPLICATE_REGISTRATION` show the message above the form. Never ask for, show or hint at an existing ID.
5. On `VALIDATION_ERROR` map `error.details` to the matching fields.

7.2 Email verification (/register/verify)

- If `registrationStore.email` is empty (page refreshed), redirect to `/register` with the message “Please register again.”
- One 6-digit field (`inputMode="numeric"`, `maxLength=6`, autofocus). Text: “We sent a 6-digit code to <email>. It expires in 10 minutes.”
- Submit calls `verify_email`. On success put `member_code` and `first_name` in `registrationStore` and navigate to `/register/success`.
- “Resend code” button calls `resend_code`. Disabled with a visible countdown for `resend_available_in_seconds`. After a `RATE_LIMITED` response show its message and disable the button.
- `CODE_INVALID`: show the message and keep the field. `CODE_EXPIRED`: show the message and offer “Send a new code”. `TOO_MANY_ATTEMPTS`: show the message and a button back to `/register`.

7.3 Success (/register/success)

- If `registrationStore.memberCode` is empty, redirect to `/register`.
- Show “Welcome, <first name>” and the 6-digit ID in very large type, with a **Copy** button (Clipboard API, with a text-selection fallback and a “Copied” confirmation).
- Show the reminder: “Save this ID now. You need it to check in to events and to see your merit points. A copy has also been emailed to you.”
- The ID exists only in memory. Do not write it to local storage, session storage or the URL. The button “I have saved my ID” clears the store and goes to the homepage.

7.4 Merit lookup (/merit)

- One 6-digit field and a “Show my points” button. Calls `merit_lookup`.
- Result: “Hello, <first name>”, total points, then two lists: events attended (title, date, points) and adjustments (date, points, reason). Show an empty-state message when a list is empty.
- `ID_NOT_FOUND` shows “ID not found.” `RATE_LIMITED` shows its message with the wait time when `retryAfter` is present.
- Do not store the ID or the result after the page is left.

7.5 Event check-in (/checkin/:eventToken, then /checkin)

This flow is the most sensitive part of the frontend. Implement it exactly as follows.

Sequence

1. **Scan.** The member scans the QR code shown on the projector with the phone camera. It opens `/checkin/:eventToken`.
2. **Device token.** `getDeviceToken()` returns a UUID from local storage, creating it with `crypto.randomUUID()` on first use. If storage is unavailable it uses an in-memory value.
3. **Position.** Call `navigator.geolocation` with `enableHighAccuracy: true, timeout: 15000, maximumAge: 0`. If the first fix has accuracy worse than 100 m, keep watching for up to 10 more seconds and use the best fix obtained.

4. **Start.** Call `start_check_in` with `event_token` (from the URL), `device_token` and `location`.
5. **Clean the address bar.** On success put `session_id`, `expires_at`, `event_title` and `points` in `checkinStore` and run `navigate("/checkin", { replace: true })`. The session id must never appear in the URL, local storage or session storage.
6. **Enter ID.** Show the event title, the notice from section 9.3, a 6-digit ID field and a countdown to `expires_at`.
7. **Submit.** Take a fresh position (same procedure as step 3) and call `check_in` with `session_id`, `member_code`, `device_token` and `location`.
8. **Result.** Show the outcome (below). On success clear `checkinStore`.

> **Required implementation detail** Navigating to `/checkin` unmounts the scan page. `checkinStore` therefore lives above the router, in a Zustand store, not in component state.

Page states

State	Screen	Next state
locating	Spinner: “Getting your location”	starting, or error
starting	Spinner: “Confirming the event code”	ready, or error
ready	ID field, notice, countdown	submitting; or expired when the countdown reaches zero
submitting	Button disabled, spinner	success, or error
success	“Checked in, +<points> points”, first name, new total, event title	End
expired	The `SESSION_INVALID` message and a hint to scan again	End
error	Message for the error code, with the action below	Retry or end

Outcomes

Result	Show	Action offered
Success	Checked in, points awarded, new total	None
`ALREADY_CHECKED_IN`	Message. Show no points	None
`CHECKIN_CLOSED`	Message	None. “Please see an executive.”
`QR_EXPIRED`, `SESSION_INVALID`	Message	“Scan the QR code on the screen again.”
`DEVICE_ALREADY_USED`	Message	“Please see an executive.”
`LOCATION_DENIED`	Message plus: “Open your browser settings, allow location for this site, then scan the QR code again.” Also shown when the browser denies permission before any call is made	“Try again” and “Please see an executive.”
`OUTSIDE_AREA`	Message	“Try again”
`ID_NOT_FOUND`	Message	Keep the form open; the member may retype the ID. The session allows 5 attempts
`RATE_LIMITED`, `SERVER_ERROR`	Message	“Try again”

A member who opens `/checkin` with no session sees: “Scan the QR code on the screen to start check-in.” Do not write coordinates, the device token or the session id to the console or to any log.

8. Admin section

All admin routes sit under `/admin`, are lazy-loaded as one chunk, and require a signed-in admin with at least one role. Hiding a menu item or button is for convenience only. RLS and the functions enforce permissions.

8.1 Authentication

Route	Behaviour
`/admin/login`	Email and password through `supabase.auth.signInWithPassword`. Link “Forgot password”. After sign-in, read roles (below). If there are none, sign out and show “This account has no access.” Otherwise go to `/admin`
`/admin/accept-invite`	Target of the invitation email. `supabase-js` reads the session from the link. Show “Set your password” (minimum 12 characters, confirm field), call `supabase.auth.updateUser({ password })`, then go to `/admin`
`/admin/reset-password`	Without a session: an email field that calls `supabase.auth.resetPasswordForEmail(email, { redirectTo: VITE_SITE_URL + "/admin/reset-password" })`. When opened from the email link (`PASSWORD_RECOVERY` event): the “Set your password” form
`/admin/*`	Guard. No session redirects to login. Sessions refresh automatically

```
// lib/roles.ts
import { jwtDecode } from 'jwt-decode';
import type { AppRole } from '../types/api-types';
export function rolesFromToken(accessToken: string): AppRole[] {
  return (jwtDecode<{ app_roles?: AppRole[] }>(accessToken).app_roles) ?? [];
}
export const hasAny = (roles: AppRole[], ...needed: AppRole[]) => needed.some(r =>
roles.includes(r));
```

Store the roles in `authStore` and refresh them on every `onAuthStateChange` event. After `set_user_role` changes the signed-in admin’s own roles, call `supabase.auth.refreshSession()`.

8.2 Roles and menu

Admin screen	Communications Head	Organization Head	Technical Head
Overview (widgets limited to what the role can read)	Yes	Yes	Yes
Registrations, export and member deletion	No	Yes (no deletion)	Yes
Events: create, edit, publish, cover image	Yes	Yes	Yes
Events: venue, radius, points, check-in window, QR, close and reset check-in	No	Yes	Yes
Events: delete	No	Yes	Yes
News	Yes	No	Yes
Attendance	No	Yes	Yes
Merit Tracker and manual adjustments	No	Yes	Yes

Admin screen	Communications Head	Organization Head	Technical Head
Media and Content	Yes	No	Yes
Messages	Yes	No	Yes
Audit log	No	No	Yes
Admins (invite, roles)	No	No	Yes

Build the menu from this matrix. Where a screen is shared (Events), disable the restricted controls for roles without access and show the tooltip “Only the Organization Head or Technical Head can change this.”

8.3 Overview (/admin)

Widget	Source	Roles
Total registered members	`admin_registrations` count	Organization Head, Technical Head
Registrations this week	`admin_registrations` count where `created_at` is in the last 7 days	Organization Head, Technical Head
Registrations per week, last 8 weeks (bar chart)	`admin_registrations` `created_at` values grouped in the browser	Organization Head, Technical Head
Latest 10 check-ins	`admin_attendance` newest first	Organization Head, Technical Head
Upcoming events	`events` count where `starts_at` is in the future	All
Draft and published news	`news_posts` counts	Communications Head, Technical Head
Unread messages	`contact_messages` count where `is_read = false`	Communications Head, Technical Head

8.4 Registrations (/admin/registrations)

- **Table** from `admin_registrations`: member ID, name, email, phone, programme, year, registered on, total points. 25 rows per page with server-side pagination (`range`), default order newest first, click a header to sort.
- **Search** (one box) matches first name, last name, email (case-insensitive contains) or exact member ID. **Filters:** programme and year.
- **Export** buttons “CSV” and “Excel”. Procedure: (1) call `log_export` with the format and the number of matching rows; (2) only if it succeeds, fetch every matching row in batches of 1,000; (3) build the file in the browser; (4) download as `spe-umat-registrations-YYYY-MM-DD.csv` or `.xlsx`. CSV uses UTF-8 with a byte-order mark so Excel opens it correctly. Phone numbers are written as text. Columns: Member ID, First name, Last name, Email, Phone, Programme, Year, Registered on, Total points.
- **Delete member** (Technical Head only): row action, confirmation naming the member, calls `delete_member`. Used for data-removal requests.

8.5 Events (/admin/events)

- **List** from `events`, newest first, with status (Draft, Published, Check-in open or closed) and actions Edit, Open QR, Delete.
- **Form fields:** title, slug (generated from the title, editable, must be unique), description (Markdown with live preview), start and end, location, cover image, published toggle. The following fields are enabled for

Organization Head and Technical Head only: points (default 3), check-in opens (defaults to the start time), check-in closes (defaults to the end time), venue, radius (default 100 m, range 20 to 1000).

- **Venue capture:** button “Use my current location”. It requests a high-accuracy position, shows latitude, longitude and the accuracy in metres, and warns when accuracy is worse than 30 m (“Move outdoors or wait for a better signal, then try again”). Save latitude and longitude with 6 decimals. Display the saved point on a static map link (`https://www.openstreetmap.org/?mlat=<lat>&mlon=<lng>`) so the admin can confirm.
- **Check-in controls** (Organization Head, Technical Head): “Open QR screen”, “Open check-in” and “Close check-in” (call `set_checkin_state`), and “Reset event secret” (confirmation, then `reset_event_secret`). “Open QR screen” is disabled until the venue and the check-in window are saved and the event is published.
- **Delete:** calls `delete_event` after confirmation. On `EVENT_HAS_CHECKINS` show its message and offer “Unpublish”.
- A published event appears on the Homepage and Events page immediately.

8.6 Live QR screen (/admin/events/:id/qr)

1. Full-screen page with a dark background and a large white QR panel, meant for a projector.
2. On load call `qr_token`. Render `VITE_SITE_URL + "/checkin/" + token` with `qrcode.react`: error correction level M, at least 70% of the screen height, white quiet zone.
3. Show the same link as text under the QR code, and the event title and venue above it.
4. Schedule the next call after `refresh_in_seconds` seconds (plus 300 ms). Show a thin progress bar for the time remaining.
5. If a refresh fails, keep the current code, retry every 3 seconds and show a “Reconnecting” badge. If no refresh has succeeded for 60 seconds, hide the code and show “Connection lost”, because the code is no longer valid.
6. Provide no download, copy-image or print control. Disable the context menu on the QR.
7. Request a screen wake lock (`navigator.wakeLock.request("screen")`) so the display does not sleep. Re-request it when the tab becomes visible.
8. Buttons “Close check-in” and “Reset secret”, each with a confirmation, and a “Back to event” link.

8.7 Attendance (/admin/attendance)

- Choose an event. List from `admin_attendance` for that event: time, member ID, name, method (QR or Manual), distance, and a count at the top.
- **Review flag:** mark in amber every QR check-in belonging to a group of 5 or more check-ins with the same `ip_hash` and `ua_family` within any 2-minute span, with the tooltip “Possible shared device or network. Review.” Nothing is blocked automatically.
- **Manual check-in:** field for the 6-digit member ID and a button. Calls `manual_check_in` and shows the member’s name and new total.
- **Remove check-in:** row action; asks for a reason (3 to 200 characters), confirms, calls `remove_check_in`.

8.8 Merit Tracker (/admin/merit)

- Member list from `admin_registrations` sorted by `total_points` descending, with search.
- Selecting a member shows the history from `admin_merit` (date, event or “Adjustment”, points, reason) and the total.
- **Adjust points:** points (non-zero integer, -100 to 100) and reason (required). A note under the field: “The reason is visible to the member.” Calls `adjust_merit`.

8.9 News (/admin/news)

- List with status and actions Edit, Publish or Unpublish, Delete.

- Form: title, slug, body (Markdown with live preview), optional cover image, published toggle. Publishing sets `published_at` to now when it is empty.
- Delete calls `delete_news_post` after confirmation.

8.10 Media and Content (/admin/media)

Tab	Behaviour
Pictures	Select several images at once. Each is resized in the browser (section 8.12), uploaded to `gallery/` and saved as a `media_items` row of type `photo` with a title (defaults to the file name), a category and an optional linked event. Grid with Delete (calls `delete_media_item`) and a published toggle
Videos and recordings	Add a title and URL, and choose YouTube or Recording. Validate YouTube links. List with Delete
Site text	Editors for `home_intro`, `about_mission`, `about_vision`, `about_benefits` and `contact_info`. Markdown with live preview. Save writes `site_content`
Calendar	The public calendar is built from published events, so it is managed on the Events screen. This tab shows a link to it

8.11 Messages, Audit log and Admins

- **Messages (/admin/messages):** list from `contact_messages`, unread first. Opening a message marks it read. A toggle marks it unread.
- **Audit log (/admin/audit):** Technical Head. Table from `audit_logs`, 50 per page, newest first, filter by action and date range. Show the admin's email by mapping `user_id` with the result of `list_admins`. Show `details` as formatted JSON in an expandable row.
- **Admins (/admin/admins):** Technical Head. Table from `list_admins` with email, roles and last sign-in. "Invite admin" form (email and role, calls `invite_admin`). Per-row buttons to add or remove a role (`set_user_role`). Show `LAST_TECHNICAL_HEAD` as an error message.

8.12 Deleting and uploading

- Every destructive action opens a confirmation dialog that names the item. Confirm is a distinct red button.
- Images: accept JPEG, PNG and WebP only, up to 5 MB before resizing. In the browser, scale the longest edge to at most 1,600 px (covers: 1,200 px) and re-encode as JPEG at quality 0.82 using a canvas (`lib/image.ts`). Reject other types with "Only JPEG, PNG or WebP images up to 5 MB are allowed."
- Show per-file progress for multiple uploads and keep going if one file fails.

9. Forms, validation and notices

9.1 Shared field rules

Implement each rule as a Zod schema in `src/schemas`. The server applies the same rules and is the final authority.

Field	Rule
Names	Trim. 1 to 50 characters
Email	Trim, lower-case, valid format, at most 254 characters
Phone	Remove spaces and hyphens. Accept `^(\+233|233|0)[2-9]\d{8}$` (Ghana) or `^\+[1-9]\d{7,14}$` (international)

Field	Rule
Programme	2 to 100 characters
Year of study	One of Year 1, Year 2, Year 3, Year 4, Postgraduate
Verification code, member ID	Exactly 6 digits. Member ID starts with 1 to 9. Use `inputMode="numeric"`, `pattern="[0-9]*"`, `autoComplete="one-time-code"` for the code
Slug	Lower-case letters, digits and hyphens, 3 to 80 characters, unique
Points per check-in	Integer 1 to 20, default 3
Radius	Integer 20 to 1000, default 100
Manual adjustment	Non-zero integer from -100 to 100; reason 3 to 200 characters
Contact message	Name 2 to 100; message 10 to 2000 characters
YouTube link	Valid URL on `youtube.com` or `youtu.be`

9.2 Form behaviour

- Validate on blur and on submit. Show the error under the field in plain language and keep what the user typed.
- Disable the submit button while a request runs. Prevent double submission.
- Map `VALIDATION_ERROR` `details` to fields by `field` name.
- Registration and Contact Us include the Turnstile widget and the hidden `website` field. Reset the widget after any failed submission.

9.3 Notices (exact wording)

Display this wording exactly. It is part of the specification.

Where	Wording
Under the registration form	The SPE UMaT Chapter collects your name, email address, phone number, programme and year of study to register you as a member, verify your email, issue your membership ID and record your merit points. Only authorised chapter executives can see this information, and it is not sold or shared. Your first name and last initial appear on the public leaderboard only if you tick the box above. Under the Data Protection Act, 2012 (Act 843) you may ask the executives to correct or remove your information.
Registration checkbox label (unticked by default)	Show my first name and last initial on the public leaderboard (optional)
Check-in page, above the ID field	Your phone's location is used only to confirm that you are at the event. It is checked and then discarded. Only the result and an approximate distance are saved.
Under the Contact Us form	Your name, email address and message are sent to the chapter executives so that they can reply to you.

10. Interface standards

- **Mobile first.** Design for a 360 px wide screen first. Test on a low-end Android phone and a recent iPhone, on a slow connection.
- **Performance.** The public bundle must stay under 250 KB gzipped. Lazy-load the admin section, charts, the lightbox and the calendar. Serve images with `loading="lazy"` and explicit width and height. Lighthouse mobile performance of 90 or more on `/` and `/checkin`.

- **Accessibility.** WCAG 2.1 AA: labelled fields, visible focus, colour contrast of 4.5:1, keyboard operation, alt text on every image (use the item title), and `aria-live` regions for form results and check-in outcomes. Lighthouse accessibility of 95 or more.
- **States.** Every data screen has loading skeletons, an empty state and an error state with a retry button.
- **Content rendering.** Markdown only through `react-markdown` with `rehype-sanitize`. Never use `dangerouslySetInnerHTML`.
- **SEO basics.** A title and meta description per public page, Open Graph image from the cover image, and `noindex` on `/admin/*`, `/checkin`, `/register/*` and `/merit`.
- **404 and errors.** A friendly 404 page and a top-level error boundary.

11. Security requirements

- No secret in the code, bundle, environment files committed to git, or the repository.
- Public pages read only the sources in section 5.1.
- The session id, QR token and device token are never logged, and the session id is never placed in the address bar, local storage or session storage.
- Coordinates are sent to the backend and then discarded. They are never stored or logged.
- The membership ID is held in memory only and only for as long as the page needs it.
- Error messages for unknown IDs are the single generic message from the error table.
- Admin routes are lazy-loaded and guarded. Permission checks in the UI are never relied on for protection.
- All user-supplied Markdown is sanitised before rendering. External links open with `rel="noopener noreferrer"`.
- Run `npm audit` before each release and pin dependency versions with the lockfile.

12. Dependencies on the backend

The backend developer delivers these, at the phase shown in section 14, so the frontend is never blocked:

- A staging project URL and anon key with seed data: published and draft events, news, media, test members with known IDs, and three test admins (one per role).
- The auth hook adding `app_roles`, and the `admin_*` views, `events_public` and `leaderboard`.
- CORS allowing `http://localhost:5173` and the Vercel preview URL on staging.
- The `site-media` bucket with its policies.
- Each Edge Function, deployed to staging, before the frontend phase that uses it.

13. Rulings

The following matters were decided in this specification. Neither developer needs to decide them again.

Matter	Ruling
API contract	The contract in section 4 is final. Request and response shapes, field names and error codes change only by a written change to both guides
Public function access	Public functions deploy with JWT verification off. Callers send the `apikey` header only. Admin functions keep JWT verification on and check roles in code
Admin role in the token	A Supabase custom access token hook adds an `app_roles` array claim, for example `["technical_head"]`. The frontend reads it for display. RLS and functions enforce

Matter	Ruling
Admin sign-in	Email and password. Public sign-up is disabled. The Technical Head invites admins through `invite_admin`. Invited admins set a password at `/admin/accept-invite`. Password reset uses `/admin/reset-password`. Minimum password length is 12 characters
First Technical Head	Created by the backend developer during setup (Backend section 3.5). All later admins are invited from the Admins screen
Permission split	As in the role matrix (section 8.2 of the Frontend guide, section 6.1 of the Backend guide). Only the Organization Head and Technical Head may export registrations
Export logging	Export is built in the browser. The frontend calls `log_export` first and builds the file only if the call succeeds
CAPTCHA	Cloudflare Turnstile on registration and Contact Us, together with a hidden honeypot field named `website`
Leaderboard	Included at `/leaderboard`. Shows the top 20 members who opted in. Opt-in checkbox on registration, unticked by default. Display name is first name and last initial
Merit adjustments	Included. Admin only, with a mandatory reason, logged. The reason is visible to the member
Event deletion	An event with check-ins cannot be deleted. Admins unpublish it instead
Check-in window	Defaults to the event's start and end time. The server accepts check-ins until 15 minutes after `checkin_closes_at`
Device lock across events	Not part of this release. The `devices` table is not created. One check-in per phone per event remains
Burst review	The server stores a hashed IP and browser family on each QR check-in. The Attendance screen flags 5 or more check-ins with the same pair within 2 minutes for review. Nothing is blocked automatically
Programme and year	Programme is a dropdown from `src/config/programmes.ts` plus "Other" with free text. Year is one of Year 1 to Year 4, or Postgraduate
Phone numbers	Ghana formats or international E.164 accepted. Stored as E.164
Email codes	6 digits, valid 10 minutes, 5 attempts, up to 3 resends with a 60-second cooldown
Data-protection wording	The wording in the notices table is final
Email sending	Resend. Sender address is the `EMAIL_FROM` secret. Staging uses Resend's test sender. Production requires the chapter's domain to be verified (prerequisite P5)
Venue privacy	Public pages read `events_public`, never `events`. Venue coordinates, radius and check-in times are never public
Rate limits	The thresholds in Backend section 7.2 are final for launch
Content format	News posts, event descriptions and site text are stored as Markdown and rendered with sanitisation
Time zone	Stored in UTC. Displayed in `Africa/Accra`
Hosting	Frontend on Vercel. Backend on Supabase. Production project on the Pro plan, which includes daily backups
Member data removal	Technical Head uses `delete_member` on request. It is logged
Contact notifications	A database webhook calls `send_contact_notification`, which emails every Communications Head and Technical Head

14. Prerequisites and build order

14.1 Prerequisites

These are accounts and materials the chapter provides. They are not design decisions. Work that depends on an item is blocked until it is supplied.

Ref	Item	Supplied by	Needed for
P1	Supabase production project (Pro plan) and a separate staging project	Technical Head	All backend work
P2	GitHub repository with the two developers as collaborators	Technical Head	All work
P3	Vercel project linked to the repository	Technical Head	Frontend deployment
P4	Domain name and DNS access	Technical Head	Production launch, email
P5	Resend account, API key, and the sending domain verified with DNS records	Technical Head	Verification emails in production
P6	Cloudflare Turnstile site (site key and secret key)	Technical Head	Registration and Contact Us
P7	Email address of the first Technical Head	Technical Head	Backend bootstrap
P8	Launch content and brand assets: logos, brand colours, photos, executives' names, positions and pictures, About text, benefits text, contact details	Communications Head	Content entry before launch

14.2 Build order

Phase	Backend delivers	Frontend delivers	Joint checkpoint
1. Foundation	Projects, schema, RLS, auth hook, storage, seed data, first Technical Head, staging	Vite app, routing, layouts, Supabase client, API client, shared types, public pages reading seed data	Public pages render from staging. Anon key cannot read private tables
2. Registration and merit	`register_member`, `verify_email`, `resend_code`, `merit_lookup`, email templates	Registration, verification, success and merit pages	A real registration end to end. Duplicate and code errors behave as specified
3. Admin core	Admin functions, invites, roles, audit log, views	Login, invite acceptance, role-gated shell, Registrations with export, Admins, Audit log	Each of the three roles sees exactly the screens in the role matrix
4. Events and check-in	`start_check_in`, `check_in`, `qr_token`, check-in database function, venue trigger	Events screen, live QR screen, check-in pages, Attendance, Merit Tracker	A full check-in in the real venue on real phones, including a refusal
5. Content and launch	Scheduled jobs, backups confirmed, production deployment, rate-limit tuning	News, Media and Content, Messages, leaderboard, performance and accessibility pass	Acceptance criteria met. Launch content loaded. Production smoke test passed

Appendix A. Database schema

> **For the frontend** Use this to generate `db-types.ts`. You read only the sources listed in section 5. Constraints, migrations and RLS belong to the backend.

This schema is final for both developers. Every table has `created_at` (default `now()`) and `updated_at` unless stated. All timestamps are `timestamptz`. Backend owns migrations and RLS. Frontend generates its types from this schema.

```
members
  id uuid PK default gen_random_uuid()
  first_name text not null, last_name text not null
  email text not null unique            -- lower case
  phone text not null unique            -- E.164
  programme text not null
  year_of_study text not null           -- Year 1 | Year 2 | Year 3 | Year 4 | Postgraduate
  member_code char(6) not null unique   -- check (member_code ~ '^[1-9][0-9]{5}$')
  show_on_leaderboard boolean not null default false
  email_verified_at timestamptz not null

email_verifications                     -- pending registrations
  id uuid PK
  email text not null unique            -- one pending row per email
  registration_data jsonb not null      -- first_name, last_name, programme,
                                        -- year_of_study, phone, show_on_leaderboard
  code_hash text not null               -- HMAC-SHA256, never the code itself
  expires_at timestamptz not null       -- 10 minutes after the latest send
  attempts integer not null default 0   -- wrong codes, maximum 5
  resend_count integer not null default 0   -- maximum 3
  last_sent_at timestamptz not null

user_roles                              -- admins only
  user_id uuid FK -> auth.users(id) on delete cascade
  role app_role not null                -- communications_head | organization_head | technical_head
  PK (user_id, role)

site_content
  key text PK                           -- home_intro, about_mission, about_vision,
                                        -- about_benefits, contact_info
  body text not null                    -- Markdown

events
  id uuid PK
  title text not null, slug text not null unique
  description text not null             -- Markdown
  starts_at timestamptz not null, ends_at timestamptz not null
  location text not null
  cover_image_url text
  points integer not null default 3
  checkin_opens_at timestamptz, checkin_closes_at timestamptz
  venue_lat numeric(9,6), venue_lng numeric(9,6)
  radius_m integer not null default 100 -- check (radius_m between 20 and 1000)
  is_published boolean not null default false

event_checkin_tokens                    -- never public; created by a trigger on events
  event_id uuid PK FK -> events(id) on delete cascade
  secret text not null                  -- 256-bit random, hex
  is_active boolean not null default true
  rotation_seconds integer not null default 30

attendance
  id uuid PK
  event_id uuid not null FK -> events(id) on delete restrict
  member_id uuid not null FK -> members(id) on delete restrict
  checked_in_at timestamptz not null default now()
  method text not null                  -- qr | manual
```

```
  device_hash text                       -- null for manual
  distance_m integer                     -- rounded to 10 m; null for manual
  ip_hash text, ua_family text           -- for review of check-in bursts; null for manual
  UNIQUE (event_id, member_id)
  UNIQUE (event_id, device_hash)

merit_records                            -- points ledger, append only
  id uuid PK
  member_id uuid not null FK -> members(id) on delete restrict
  event_id uuid FK -> events(id) on delete restrict
  attendance_id uuid FK -> attendance(id) on delete set null
  points integer not null
  source text not null                   -- event_checkin | manual_adjustment | check_in_reversal
  reason text
  awarded_by uuid FK -> auth.users(id)

news_posts
  id uuid PK
  title text not null, slug text not null unique
  body text not null                     -- Markdown
  cover_image_url text
  is_published boolean not null default false
  published_at timestamptz
  created_by uuid FK -> auth.users(id)

media_items
  id uuid PK
  type text not null                     -- youtube | recording | photo
  title text not null
  category text not null default 'general'  -- photos: general | event | activity | member |
executive
  url text                               -- YouTube or recording link; null for photos
  storage_path text                      -- path in bucket site-media; null for links
  event_id uuid FK -> events(id) on delete set null
  is_published boolean not null default true

contact_messages
  id uuid PK
  name text not null, email text not null, message text not null
  is_read boolean not null default false

audit_logs                               -- append only
  id uuid PK
  user_id uuid FK -> auth.users(id)
  action text not null
  details jsonb not null default '{}'
  ip_address inet
  created_at timestamptz not null default now()

checkin_sessions
  id uuid PK default gen_random_uuid()
  event_id uuid not null FK -> events(id) on delete cascade
  device_hash text not null
  expires_at timestamptz not null        -- 3 minutes after the scan
  used_at timestamptz
  attempts integer not null default 0    -- maximum 5

rate_limits
  key text PK
  window_start timestamptz not null
  count integer not null
```

Views

View	Readable by	Columns
`events_public`	Everyone	`id, title, slug, description, starts_at, ends_at, location, cover_image_url, points` for published events only. Hides venue, radius and check-in times
`leaderboard`	Everyone	`display_name` (first name and last initial, for example "Kwame A."), `total_points`, `rank`. Only members with `show_on_leaderboard = true`. Ranked among those members
`admin_registrations`	Organization Head, Technical Head	`id, first_name, last_name, email, phone, programme, year_of_study, member_code, show_on_leaderboard, created_at, total_points`
`admin_attendance`	Organization Head, Technical Head	`attendance_id, event_id, member_id, member_code, first_name, last_name, checked_in_at, method, distance_m, ip_hash, ua_family`
`admin_merit`	Organization Head, Technical Head	`id, member_id, member_code, first_name, last_name, event_id, event_title, points, source, reason, awarded_by, created_at`

The three `admin_*` views are created with `security_invoker = true`, so RLS on the base tables applies to the caller. `events_public` and `leaderboard` run as the view owner and are granted to `anon`.

Appendix B. API types

Copy this file to `src/types/api-types.ts` unchanged.

```
// api-types.ts  (identical file in the frontend repo; backend validates with matching Zod schemas)
export type ErrorCode =
  | 'VALIDATION_ERROR' | 'CAPTCHA_FAILED' | 'CODE_INVALID' | 'LOCATION_DENIED'
  | 'UNAUTHORIZED' | 'FORBIDDEN' | 'CHECKIN_CLOSED' | 'OUTSIDE_AREA' | 'ID_NOT_FOUND'
  | 'NOT_FOUND' | 'DUPLICATE_REGISTRATION' | 'ALREADY_CHECKED_IN' | 'DEVICE_ALREADY_USED'
  | 'EVENT_HAS_CHECKINS' | 'LAST_TECHNICAL_HEAD' | 'CODE_EXPIRED' | 'QR_EXPIRED'
  | 'SESSION_INVALID' | 'RATE_LIMITED' | 'TOO_MANY_ATTEMPTS' | 'SERVER_ERROR';

export type ApiResponse<T> =
  | { ok: true; data: T }
  | { ok: false; error: { code: ErrorCode; message: string;
      details?: { field: string; message: string }[]; retry_after_seconds?: number } };

export type AppRole = 'communications_head' | 'organization_head' | 'technical_head';
export type YearOfStudy = 'Year 1' | 'Year 2' | 'Year 3' | 'Year 4' | 'Postgraduate';
export interface Location { lat: number; lng: number; accuracy: number }

export interface RegisterMemberReq {
  first_name: string; last_name: string; programme: string; year_of_study: YearOfStudy;
  email: string; phone: string; show_on_leaderboard?: boolean;
  captcha_token: string; website: string;
}
export interface RegisterMemberRes { email: string; expires_in_seconds: number;
resend_available_in_seconds: number }
export interface VerifyEmailReq { email: string; code: string }
export interface VerifyEmailRes { member_code: string; first_name: string }
export interface ResendCodeReq { email: string }
export type ResendCodeRes = Omit<RegisterMemberRes, 'email'>;

export interface MeritLookupReq { member_code: string }
export interface MeritLookupRes {
  first_name: string; total_points: number;
  attendance: { event_title: string; event_date: string; points: number; checked_in_at: string }[];
  adjustments: { points: number; reason: string; created_at: string }[];
```

```
}

export interface StartCheckInReq { event_token: string; device_token: string; location: Location }
export interface StartCheckInRes { session_id: string; expires_at: string; event_title: string;
points: number }
export interface CheckInReq { session_id: string; member_code: string; device_token: string;
location: Location }
export interface CheckInRes { first_name: string; points_awarded: number; total_points: number;
event_title: string }

export interface SubmitContactReq { name: string; email: string; message: string; captcha_token:
string; website: string }

export interface QrTokenReq { event_id: string }
export interface QrTokenRes { token: string; refresh_in_seconds: number }
export interface SetCheckinStateReq { event_id: string; is_active: boolean }
export interface ManualCheckInReq { event_id: string; member_code: string }
export type ManualCheckInRes = Omit<CheckInRes, 'event_title'>;
export interface RemoveCheckInReq { attendance_id: string; reason: string }
export interface AdjustMeritReq { member_code: string; points: number; reason: string }
export interface LogExportReq { format: 'csv' | 'xlsx'; row_count: number }
export interface AdminSummary { user_id: string; email: string; roles: AppRole[]; last_sign_in_at:
string | null }
export interface InviteAdminReq { email: string; role: AppRole }
export interface SetUserRoleReq { user_id: string; role: AppRole; action: 'add' | 'remove' }
```